

# DARK LATTICE

---

Systems that react to each other, not  
events we hand-placed.

THREE GAMES • ONE ACCUMULATING SYSTEMS STACK

ENTER THE SYSTEM ↓

**WE DO NOT BUILD  
DIFFICULTY FROM  
MORE HAND-PLACED  
EVENTS.**

**WE BUILD RULES  
THAT CREATE  
CONSEQUENCES  
TOGETHER.**

In most games, the difficulty is placed by hand. A designer decides an enemy stands here, a trap fires there, this consequence happens at that moment. It works — but somebody has to build every situation individually, so the game can only ever contain the situations there was time to build.

**We write the rules instead.** We decide how the pieces behave and how they affect each other, then let them run. The hard moments come out of those rules colliding — situations nobody placed, and nobody could have placed.

This is not the same as making it random. Every outcome follows from rules we wrote, so we can test it, repeat it, and show you exactly why it happened.

It matters commercially because of what it leaves behind. A hand-built game is worth roughly what is in it. A rules-based one also leaves the rules, the tests that prove them, and the knowledge of how to build them again — and that is what carries into the next game.

# DESK42

PERSISTENT SYSTEMS PROOF

---

You work a desk inside a bureaucracy, deciding people's claims.

What you decide does not disappear when the case closes. A ruling you make in your first week shapes what lands on your desk in your sixth. The system keeps its own record of what really happened, separately from what was officially written down — and those two things drift apart.

Nothing here is random. Every consequence traces back to a decision you made, and the game can show you the chain.

## ENGINE

Unity

---

## PLATFORM

Windows PC, single player

---

## DISTRIBUTION

Steam PC first

---

## STATUS

Bounded causal slice, not a finished product

---

---

PERSISTS INTO

# BRAWLER

REACTIVE SIMULATION

---

A fighting game where the ground itself is alive.

Spread something across the floor and it keeps spreading — into enemies, into you, into the next room. It is not an effect painted over the fight. It decides where you can stand, what hurts you, and what your attacks do.

So you stop treating the room as scenery and start using it. **You change the environment, the environment changes the fight.** The maths behind it is well established and we do not claim to own it. What is ours is the implementation and the tools around it.

## ENGINE

Unreal Engine

---

## SYSTEM

Deterministic reactive field

---

## BASIS

Established reaction-diffusion methods

---

## STATUS

In development; benchmarks not published

---

---

COMBINES INTO

# ROGUELITE

PERSISTENT SYSTEMS + REACTIVE SIMULATION

---

A possible third game that runs both systems at once.

A world that remembers what you did, inside an environment that reacts while you are still standing in it.

**It is not committed.** We build it only if the first two games show us it is worth building — what they cost, how long they took, and whether anyone wants it. We would rather say that plainly than put a third title on a roadmap to make the plan look bigger.

## STATUS

Roadmap direction

---

## COMMITMENT

Conditional on prior evidence

---

## DEPENDENCY

Not a base-case revenue line

---

# THE REUSABLE SYSTEMS FOUNDATION

---

## 01 SOURCE CODE

---

## 02 SYSTEM DESIGN

---

## 03 FOUNDER KNOW-HOW

---

### 01 — SOURCE CODE

The code itself — and the least of the three.

Plain logic travels reasonably well: the maths, the rules, the decision-making, the data handling. Anything written against a particular engine's objects and tooling does not. That gets rewritten.

**We size this honestly.** For gameplay code tied closely to an engine, very little moves across.

---

VIEW CROSS-ENGINE EVIDENCE



Practical carry-over between Unity and Unreal, by area.  
These are practical ranges, not guaranteed engineering  
measurements. They describe a Unity-to-Unreal transition

between two products, not a claim that any figure has been benchmarked.

UNITY  
→  
UNREAL  
CARRY-  
OVER

#### Engine-specific gameplay code

CARRY-OVER 0-20%

---

#### Algorithms and system logic

CARRY-OVER 80-100%

---

#### Behaviour specifications

CARRY-OVER 80-100%

---

#### Test cases / acceptance criteria

CARRY-OVER 70-100%

---

#### Engine-independent game data

CARRY-OVER 80-100%

---

#### Raw art assets

CARRY-OVER 70-100%

---

#### Engine-specific shaders / VFX / UI

CARRY-OVER 10-50%

---

#### Audio source assets

CARRY-OVER 90-100%

---

#### Production workflows

CARRY-OVER 70-100%



---

Commercial knowledge

CARRY-OVER 90-100%

---

Company-owned IP

CARRY-OVER 100% in principle

---

# CAPABILITY THAT ACCUMULATES

01

## A capability that accumulates

Anyone can copy the idea. The idea is one sentence long.

What is slow to copy is everything underneath it: the architecture, the tools that check it works, the hundred decisions that made it behave. Each of those is cheap to describe and expensive to build — and they stack. The longer we run, the wider the gap between describing this work and reproducing it.

02

## Every game leaves infrastructure behind

Each game hands the next one a head start: how these systems should behave, what broke last time, how to test them, how long production really takes, how to get a game in front of people.

That holds even when the engine changes and the code is rewritten from scratch.

03

# Designed to ship commercially

We sell games. Paid PC games, downloaded, starting with Desk42 on Steam and then Brawler. That is the whole business model.

We are not selling an engine, licensing technology, chasing a patent or waiting on a grant. The reusable technology is how we keep our own costs down — **it is not a product**, and nothing in the plan depends on the third game being made.

# WHAT WE CAN SUPPORT WHAT TRANSFERS — AND WHAT DOES NOT

Everything above is what we are building. This is the part we can prove, and the part we cannot. If a claim is not in the left column, we are not making it. We would rather be checked than believed.

---

## WHAT THE EVIDENCE SUPPORTS

---

- 01 **Systems and specifications carry between engines; code often does not.** Rules, state definitions, data structures, equations, edge cases, expected behaviour, test cases and architecture decisions survive an engine change. Engine-specific gameplay code is assessed separately and largely rewritten.
- 02 **Desk42 is a single-player Windows PC game built in Unity.** The player operates inside an institutional claims system where evidence, policy, procedure, adjudication, queues, appeals and precedent create persistent consequences. Distribution is Steam PC first.
- 03 **Brawler is an action roguelite in development in Unreal Engine.** Its system-level proposition is a spatial simulation whose state is authoritative for movement, hazards, actor exposure and combat consequences, rather than a cosmetic effect layer.
- 04 **The reaction-diffusion methods are established; the implementation is ours.** We claim a gameplay implementation and toolchain built on established reaction-diffusion methods — not ownership of the underlying mathematics.
- 05 **Founder capability is inspectable in public.** SatyBT is a founder-created behaviour-tree framework published under the MIT licence. It stands as evidence that we can design, test, document, package and maintain reusable technical infrastructure.
- 06 **The commercial model is deliberately narrow.** Premium downloadable PC games through digital distribution, beginning with Desk42 and followed by Brawler. Reusable technology is an internal margin and execution advantage, not a product.

- 01 **Not one proprietary engine.** We do not sell, license or claim a named proprietary game engine. There is no separate customer, package, price, sales channel or licence revenue in the base case.
- 02 **Not direct code reuse across engines.** We do not claim that every line of code transfers between Unity and Unreal, and we do not claim that changing engine is cheap.
- 03 **Not a finished product.** The current Desk42 implementation is a bounded causal slice. It is not described as near-complete, production-ready or market-validated.
- 04 **No performance figures until measured.** No runtime benchmark, frame budget or simulation-scale figure is published here. Those follow a frozen build and recorded measurements, not an estimate.
- 05 **No market validation yet.** External playtests, store-page behaviour, wishlist data and purchase-intent work have not been completed. Comparable titles establish plausible categories and price bands; they do not evidence our demand.
- 06 **No price, date or scale commitments.** No release date, sales figure, final price, team size, partner, award or press claim appears on this site, because none is currently supported by inspected evidence.

---

Dark Lattice does not depend on copying one engine-specific codebase across every product. Its scalability comes from accumulating reusable technical knowledge, system architecture, proprietary implementations, production infrastructure, IP and market evidence across multiple releases. Where engine-specific code must be rewritten, the underlying problem has already been designed, tested and understood, reducing repeated discovery and development risk.

# DARK LATTICE

`darklattice.co.uk`

---

DARK LATTICE – SYSTEMS STUDIO

THREE GAMES • ONE ACCUMULATING SYSTEMS STACK